Demo: LLM-Based Browser Extension for Phishing Detection

Eric Pham
Department of Engineering
San Jose State University, CA, USA
eric.p.pham@sjsu.edu

Thinh Bui
Department of Engineering
San Jose State University, CA, USA
thinh.bui@sjsu.edu

Haoming Chen
Department of Engineering
San Jose State University, CA, USA
haoming.chen@sjsu.edu

2025 Silicon Valley Cybersecurity Conference (SVCC) | 979-8-3315-3429-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/SVCC65277.2025.11133657

***Abstract*—Phishing emails and scam messages pose a significant cybersecurity threat to the public. In this project, we implement a new method of detection and prevention of these cybersecurity threats by using the latest large language models (LLMs). LLMs, like OpenAI, excel in performing text-related tasks, including in-depth sentiment and behavioral analysis of texts. By integrating these LLM features into a browser extension, we aim to provide a user-friendly and accessible threat detection solution that can protect users from phishing attacks.**

***Index Terms*—Cybersecurity, Large Language Model, OpenAI GPT-4, Phishing Detection, Prompt Engineering**

I. Introduction

Phishing attacks like emails or spam messages have always been a cybersecurity threat, and they continue to evolve into new and sophisticated forms. Phishing attacks are socially engineered attacks that exploit human cognitive biases and decision-making shortcuts [1]. Unlike traditional malware, which can be detected and blocked by programs, these threats are far more effective in disguising themselves from detection and have a greater potential to cause harm to unsuspecting people. Consequently, phishing attacks remain one of the most effective methods for compromising systems and stealing private data, posing a serious challenge to organizations and individuals [1]. Emails are still one of the most commonly used forms of communication and are commonly associated with a variety of phishing attacks. Recently, as Wash states, "technical phishing tools are able to detect many phishing URLs … [but] they work best for mass phishing emails" [2]. Mass phishing emails are the most common type of phishing, but different forms like spear-phishing are becoming more common as they are more customized to a smaller group of targets.

As LLMs continue to evolve, they can help aid in phishing detection and are better suited compared to traditional machine learning (ML) models. Traditional ML models typically use feature engineering for phishing detection and are very selective with their features to prevent general phishing. However, the models often struggle to detect spear-phishing attacks since they are more personalized and catered to their targets. LLMs can help address spear-phishing attacks that would go undetected in traditional ML models since they can analyze message tones, behaviors, and subtle inconsistencies for a more accurate detection that would go undetected with traditional models.

II. Related Work

Early phishing email detection systems relied heavily on browser-based solutions such as toolbars (e.g., SpoofGuard and Netcraft). These solutions usually lacked access to critical contextual information found in emails, such as sender info, email headers, bodies, and language. To address this issue, PILFER, a machine learning based phishing detection system proposed by Fette et al., classifies emails based on carefully selected sets of phishing-targeted features extracted from the content of the emails and their external sources. Features extracted from the emails, ranging from abnormal URLs and domain names to hidden JavaScripts and redirects, were extracted and applied in both training and testing. Multiple classifiers, including SVM, decision trees, and Bayesian, are tested to implement the classification, and Random Forest was determined to be the best performer. The random forest approach achieved high accuracy in detecting phishing emails with minimal false positives, highlighting the value of context-aware and feature-rich models in email-based phishing detection [3].

Recently, the popularity of LLMs has skyrocketed, and they are often utilized in our everyday lives. Regarding phishing detection, many have experimented with various LLMs to evaluate their effectiveness in detecting phishing. It was found that "the OpenAI-developed models excelled in identifying phishing emails," compared to other models like Llama, Claude, and Mistral [4].

III. Proposed method

The proposed method is a phishing detection browser extension that will analyze email content for phishing and classify it with safe, suspicious, or phishing labels using OpenAI's API. The application is made up of three main components: Frontend, Backend, and the LLM classification. In the following subsections, we elaborate on what each component is in charge of and the tools/technologies that were used to build them.

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 09:09:32 UTC from IEEE Xplore. Restrictions apply.

A. Frontend (Browser Extension UI)

For the frontend, we are using React.js to develop the browser extension that will capture the email data and present the classification results to the user. The frontend's main purpose is to display the model's classification result and provide a simple and easy user experience.

The general flow of the application is that the frontend will capture all the relevant email data and send it to the backend for processing. After the backend processes the email and sends it to the LLM, it will send it back to the frontend, where it will display the classification results and justification.

B. Backend (Flask API)

For the backend, we will be using Flask for the implementation. The backend will serve as the bridge between the frontend and the LLM. Its main purpose is to handle the preprocessing and communication with the LLM. In terms of data preprocessing, we want to extract key information from each email, like the sender's email address, file attachments, and website links/URLs. We also plan on implementing more preprocessing steps by looking for phishing signals such as shortened URLs, mismatching information, urgent language, and poor grammar/formatting. The backend will also utilize prompt engineering to give the LLM model for classification. In another study where the GPT-4 model was used, it was found that the "output is sometimes inconsistent, so we enhance the classification efficiency by careful design of the prompt" [5]. By doing so, we can improve the model's effectiveness and tailor it to our specific design.

C. LLM Classification

For the LLM in our proposed method, we will be using Google's API Gemini-2.0-flash [6] model to analyze the emails' content and provide the classification as well as its justification. Even though OpenAI's GPT-4 model has the best performance compared to other LLMs, accessing the free-tier OpenAI API took months to complete. Thus, we choose Google Gemini's API as the LLM model for the browser extension. From the backend, the content of the emails, like sender address, full text/content of the email, and other extracted metadata, will be sent to the model. Google's Gemini model will use all the information provided by the backend, and utilizing prompt engineering, the data will go through a custom prompt that will be fine-tuned with our evaluation metrics. Once it has gone through the custom prompt, the model will provide a classification label (safe, suspicious, or phishing) and a justification for why the label was assigned. With Google's model, they can provide the user with information about the signals found in the email.

IV. Evaluation

A. Finetuning LLM Prompt/Results

1. *Test data*: We use the public phishing email dataset from [7] for our experiments. The phishing email dataset in [7] contains six different datasets obtained from various sources. For the LLM tuning task, the CEAS sub-dataset from [7] is chosen since it is the largest dataset and has various features that can be used for testing. The CEAS dataset is in CSV format with nearly 40000 records collected from 2008 to 2010, each containing the email information such as: sender's email, receiver's email, subjects, body, number of URLs in the email, and the label value. The label value 0 represents the non-phishing email, while the label value 1 represents the phishing email. The percentage of phishing emails in the dataset is 50%. For the experiment, since we experienced free-resource limitations of Gemini's API, only a sample of 200 records from the CEAS dataset was used for testing purposes.
2. *Prompt design*: For the phishing email detection, the LLM model is considered as a classifier model, which takes the prompt instruction and labels the given email as phishing or non-phishing. There are several strategies to design prompts, such as zero-shot learning, few-shot learning, chain-of-thoughts, etc. For our proposed method, we employed zero-shot learning due to its simplicity, yet it achieves remarkable results, as shown in the next section. In the zero-shot prompt, we rely on the ability of performing tasks without additional examples of LLM models. Its instruction consists of questions and input values. The prompt's question asks the LLM model to classify the email as phishing or non-phishing, while the input values of the prompt are values extracted from the CSV files, including subject, body, etc.
3. *Prompt testing results*: For the experiments, we tried to query the LLM according to four different input value selections. We named the selections as body_only, body_extracted_url, body_without_url, and body_with_sender. Details about the email's features in each selection can be found in Table 1.

Selection	Email feature used	Accuracy
body_only	subject, body	90%
body_extracted_url	Subject, body, URLs extracted from body	90%

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 09:09:32 UTC from IEEE Xplore. Restrictions apply.

body_without_url	Subject, body with URLs removed	90%
body_with_sender	Subject, body, sender	94%

Table 1. Input value selections

Each input selection will be tested on a dataset of 100 phishing emails and 100 non-phishing emails extracted from the CEAS dataset. The accuracy of predictions for each input selection is recorded in Table 2.

From Table 1, it is also clear that the input with the sender email outperforms the other input selections in the phishing classification task, and it would be beneficial if the browser extension could capture the sender email. In contrast, the URL in the email's body does not improve the accuracy, the browser extension can safely skip extracting the embedded URL from the email's website. This finding helps us reduce the time implementing the browser extension since extracting embedded URLs from email is not a trivial task.

B. LLM model evaluation

For the proposed method, we want to assess and evaluate the performance of the model. We used the SpamAssassin dataset [8] to help evaluate and quantify the model's performance. The metrics we will use include:

1. *Accuracy:* Represents the overall proportion of correctly classified emails (both phishing and legitimate) out of all the samples.
2. *Precision:* Measures the proportion of correctly identified phishing emails out of all emails the model flagged as phishing.
3. *F1-score:* The harmonic mean of precision and recall. It provides a balanced measure when both false positives and false negatives are important.

With these evaluation metrics, we wanted to reduce as many false negatives since we would rather have legitimate emails be marked as phishing and not phishing emails be marked as legitimate. Using 100 samples from the SpamAssassin dataset, we were able to evaluate the model's performance as shown in Table 2. The model achieved an accuracy of 91%, producing 5 true negatives and 4 false positives.

	Actual Phishing	Actual Legitimate
Predicted Phishing	46	4
Predicted Legitimate	5	45

Table 2. Confusion matrix for the model's performance on the dataset

The precision of the model is 0.92, while the F1-score of the model is 0.91. These results show that the model was able to perform well and accurately determine whether an email was phishing or not.

V. Conclusion and Future Work

We were able to create a browser extension that automates phishing detection in emails so that users can better protect themselves from one of the most common cyber attacks through Google's Gemini-2.0-flash model. With our proposed method, we were able to combine prompt engineering, key signal extraction, and the LLM to provide a classification label as well as a justification.

For future implementation, we plan on improving the preprocessing and email extraction through fine-tuning. An ensemble voting from different LLM models could be used to boost the prediction's accuracy. We also want to improve the UI/UX of our prototype by adding new features such as scan history or scanning multiple emails. Potentially, we hope to implement multi-browser support.

References

[1] K. Singh, P. Aggarwal, P. Rajivan, and C. Gonzalez, "What makes phishing emails hard for humans to detect?," *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 64, no. 1, pp. 431–435, Dec. 2020, doi: https://doi.org/10.1177/1071181320641097.

[2] R. Wash, "How Experts Detect Phishing Scam Emails," Proceedings of the ACM on Human-Computer Interaction, vol. 4, no. CSCW2, pp. 1–28, Oct. 2020, doi: https://doi.org/10.1145/3415231.

[3] I. Fette, N. Sadeh, and A. Tomasic, "Learning to detect phishing emails," in Proceedings of the 16th International Conference on World Wide Web, 2007, pp. 649–656.

[4] H. Patel, U. Rehman, and F. Iqbal, "Evaluating the Efficacy of Large Language Models in Identifying Phishing Attempts," arXiv.org, Apr. 23, 2024. https://arxiv.org/abs/2404.15485

[5] S. Si, Y. Wu, L. Tang, Y. Zhang, J. Wosik, and Q. Su, "Evaluating the Performance of ChatGPT for Spam Email Detection," arXiv.org, 2024. https://arxiv.org/abs/2402.15537

[6] "Gemini 2.0 flash" | Generative AI on Vertex AI | Google Cloud,"Google,https://cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-0-flash (accessed May 2, 2025).

[7] A. Al-Subaiey, M. Al-Thani, N. A. Alam, K. F. Antora, A. Khandakar, and S. A. U. Zaman, "Novel Interpretable and Robust Web-based AI Platform for Phishing Email Detection," *arXiv.org*, May 19, 2024. https://arxiv.org/abs/2405.11619

[8] "Phishing Email Dataset," *www.kaggle.com*. https://www.kaggle.com/datasets/naserabdullahalam/phishing-email-dataset

Appendix

A presentation and final demo are available at this YouTube link: https://youtu.be/vzpTbR60RJg

The source code for this project is available at this link: https://github.com/Xainax/CMPE209FinalProject

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 09:09:32 UTC from IEEE Xplore. Restrictions apply.